

# CPSY 300 Project 1: Cloud-Native Nutritional Insights

Team: Yuandong Yang (000949205), Ethan Bayarsaikhan (000961411), Justin Norman-Rance  
Report generated: 2026-10-05 14:37 Mountain Daylight Time

| Task   | Deliverable file                        |
|--------|-----------------------------------------|
| Task 1 | data_analysis.py                        |
| Task 2 | Dockerfile                              |
| Task 3 | lambda_function.py                      |
| Task 4 | .github/workflows/deploy.yml            |
| Task 5 | 1-page enhancement report (in this PDF) |

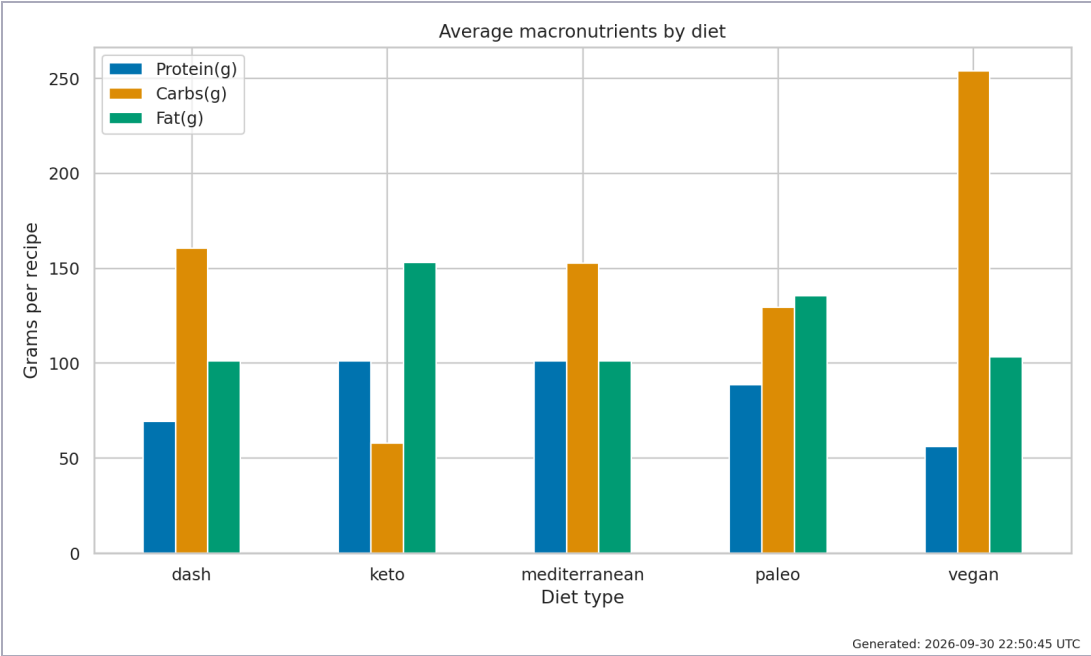
Each section below contains the screenshots the assignment requires for that task; every caption gives the date and time visible in the screenshot.

# Task 1: Dataset Analysis and Insights

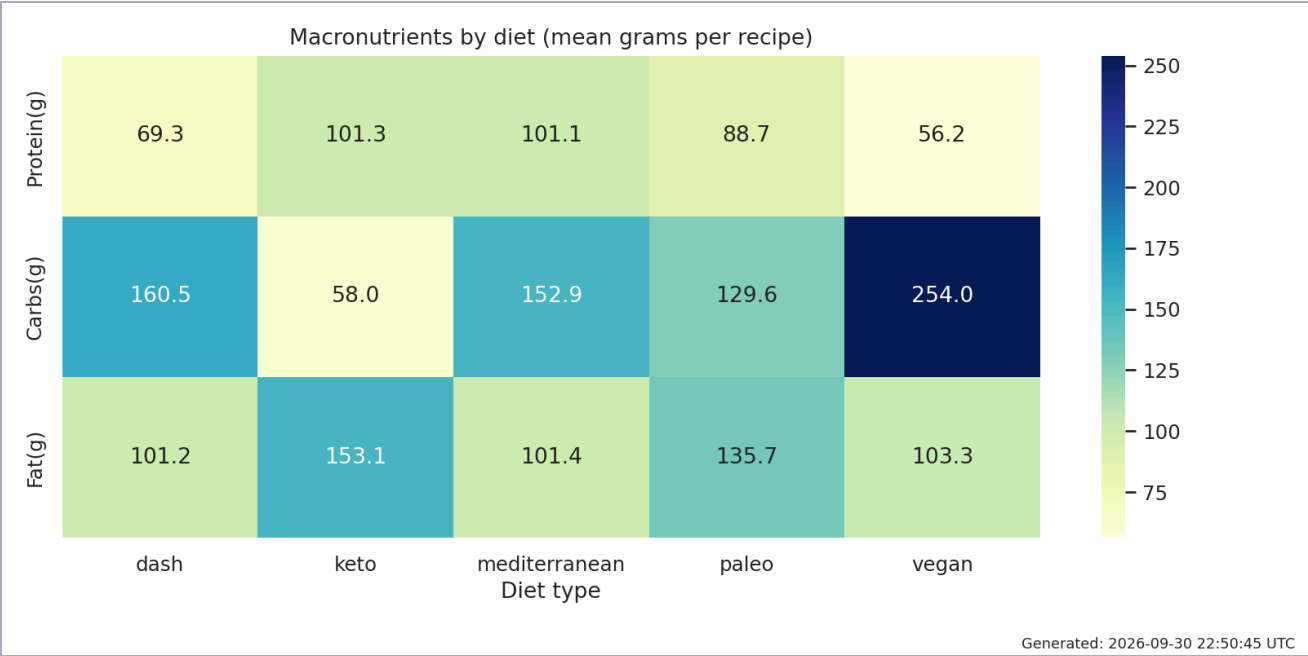
Deliverable submitted in the zip: `data_analysis.py`

```
Activities Terminal Wed Sep 30 16:53:01
CPSY 300 Project 1 | Task 1 | Saved execution log and output
2026-09-30T16:52:59-06:00
Container nutritional-insights-analysis-run-e8bc1bc8a8ab Creating
Container nutritional-insights-analysis-run-e8bc1bc8a8ab Created
2026-09-30 22:50:45 UTC
Protein(g) Carbs(g) Fat(g)
Diet_type
dash 69.28 160.54 101.15
keto 101.27 57.97 153.12
mediterranean 101.11 152.91 101.42
paleo 88.67 129.55 135.67
vegan 56.16 254.00 103.30
Most common cuisines:
Diet_type Cuisine_type count
dash american 639
keto american 663
mediterranean mediterranean 1274
paleo american 535
vegan american 925
{
  "generated_at": "2026-09-30 22:50:45 UTC",
  rows : 7806
  diet_types : 5
  highest_mean_protein_diets : ['keto']
  highest_mean_protein_g : 101.26650793650794
  highest_total_protein_diets : ['mediterranean']
  highest_total_protein_g : 177249.89
  undefined_ratios : {'Protein_to_Carbs_ratio': 0, 'Carbs_to_Fat_ratio': 38}
```

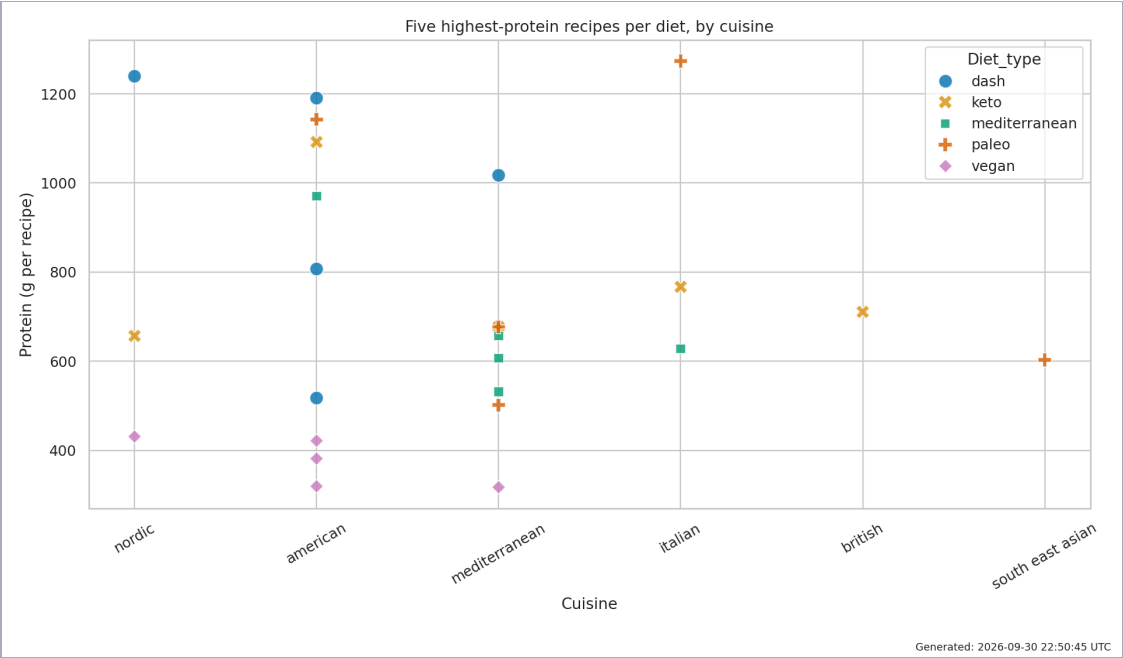
Data processing and calculations in Python (Pandas), run in Docker on the Ubuntu VM: average macronutrients per diet, most common cuisines, highest-protein diet, undefined ratio counts. Clock: Wed Sep 30 16:53:01.



Bar chart: average protein, carbs and fat per diet type (generation timestamp bottom right).



Heatmap: macronutrient content by diet type (generation timestamp bottom right).



Scatter plot: the top 5 protein-rich recipes per diet type by cuisine (generation timestamp bottom right).

# Task 2: Dockerizing the Data Processing Application

Deliverable submitted in the zip: [Dockerfile](#)

```
es Terminal Fri Oct 2 10:08:25 Task2-docker-run
2026-10-02 16:06:45 UTC
Diet_type Protein(g) Carbs(g) Fat(g)
dash 69.28 160.54 101.15
keto 101.27 57.97 153.12
mediterranean 101.11 152.91 101.42
paleo 88.67 129.55 135.67
vegan 56.16 254.00 103.30
Most common cuisines:
Diet_type Cuisine_type count
dash american 639
keto american 663
mediterranean mediterranean 1274
paleo american 535
vegan american 925
{
  "generated at": "2026-10-02 16:06:45 UTC",
  "rows": 7806,
  "diet types": 5,
  "imputed values": {
    "Protein(g)": 0,
    "Carbs(g)": 0,
    "Fat(g)": 0
  },
  "highest_mean_protein_diets": [
    "keto"
  ],
  "highest_mean_protein_g": 101.26650793650794,
  "highest_total_protein_diets": [
    "mediterranean"
  ],
  "highest_total_protein_g": 177249.89,
  "undefined_ratios": {
    "Protein to Carbs_ratio": 0,
    "Carbs to Fat_ratio": 38
  },
  "interpretation": "Recipe-level values; no serving size or medical inference."
}
Saved tables and three charts to /app/output
--- container exited OK at 2026-10-02 10:06:46 MDT ---
```

Docker container running locally and processing the data: 7,806 recipes processed, averages and summary printed. Clock: Fri Oct 2 10:08.

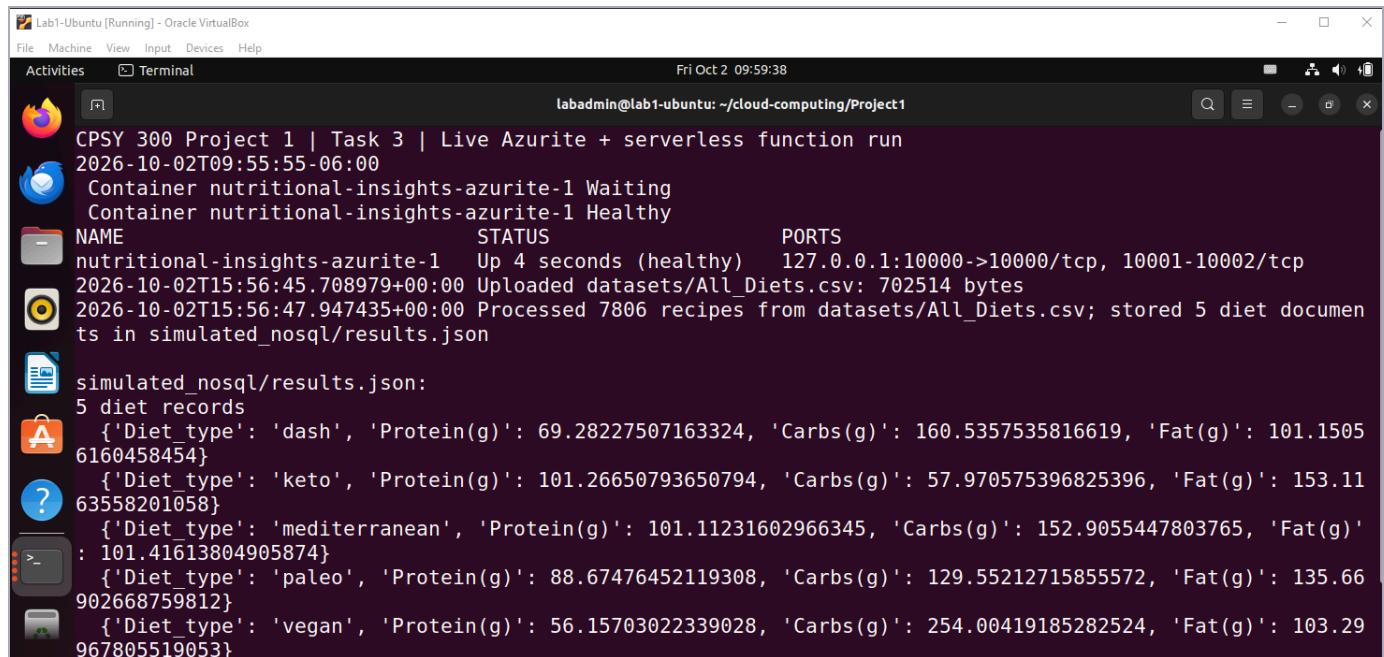
```
Activities Terminal Wed Sep 30 16:53:04
CPSY 300 Project 1 | Task 2 | Live services and registry logs
2026-09-30T16:53:01-06:00
NAME PORTS IMAGE COMMAND SERVICE CREATED STATUS
nutritional-insights-azurite-1 mcr.microsoft.com/azure-storage/azurite:3.35.0 "docker-entrypoint.s..." azurite 6 minutes ago Up 6 m
inutes (healthy) 127.0.0.1:10000->10000/tcp, 10001-10002/tcp
nutritional-insights-registry-1 registry:2.8.3 "/entrypoint.sh /etc..." registry 5 minutes ago Up 5 m
inutes 127.0.0.1:5001->5000/tcp
Info -> U In Use
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
diet-analysis:baseline 91d9d3648d81 698MB 193MB
diet-analysis:project1 f5595b07f723 568MB 130MB U
diet-analysis:tests 0dd76a2c1444 575MB 131MB
Saved docker push result:
6b37362b3da7: Pushed
2a8acfc0df46: Pushed
project1: digest: sha256:f5595b07f72304299087700bbc2459eed3fe069d3267db0c066a334b177ba8f1 size: 856
Saved docker pull result:
project1: Pulling from diet-analysis
Digest: sha256:f5595b07f72304299087700bbc2459eed3fe069d3267db0c066a334b177ba8f1
Status: Image is up to date for localhost:5001/diet-analysis:project1
localhost:5001/diet-analysis:project1
```

Deployment simulated with Docker Compose (Azurite and registry services up), and the image pushed to and pulled from a local registry (localhost:5001) with the same digest. Clock: Wed Sep 30 16:53:04.

### Task 3: Serverless Data Processing with Azurite

Deliverable submitted in the zip: `lambda_function.py` · Prepared by Yuandong Yang

**How cloud storage and serverless processing were simulated.** Azurite, Microsoft's local Azure Storage emulator, runs as a Docker Compose service and exposes the Blob service at `http://127.0.0.1:10000/devstoreaccount1` with the emulator's public development account, so no real Azure resources are used. `upload_dataset.py` creates the `datasets` container and uploads `All_Diets.csv` with the Azure Blob Storage SDK (by script rather than Azure Storage Explorer); the SDK calls are the same as against real Azure, only the connection string points to Azurite, and `storage_config.py` refuses any non-local endpoint. `lambda_function.py` is the simulated serverless function: when invoked it downloads the blob with `BlobServiceClient`, cleans the data, calculates the average protein, carbs and fat per diet type, and writes one JSON document per diet to `simulated_nosql/results.json`, which stands in for a NoSQL database such as Cosmos DB. The file is replaced atomically, so a failed run never leaves a partial result. Azurite has no event triggers, so the function is invoked manually after each upload (`docker compose run function`), the local equivalent of a blob-triggered Azure Function.



```
Lab1-Ubuntu [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Terminal
labadmin@lab1-ubuntu: ~/cloud-computing/Project1
CPSY 300 Project 1 | Task 3 | Live Azurite + serverless function run
2026-10-02T09:55:55-06:00
Container nutritional-insights-azurite-1 Waiting
Container nutritional-insights-azurite-1 Healthy
NAME STATUS PORTS
nutritional-insights-azurite-1 Up 4 seconds (healthy) 127.0.0.1:10000->10000/tcp, 10001-10002/tcp
2026-10-02T15:56:45.708979+00:00 Uploaded datasets/All_Diets.csv: 702514 bytes
2026-10-02T15:56:47.947435+00:00 Processed 7806 recipes from datasets/All_Diets.csv; stored 5 diet documents in simulated_nosql/results.json
simulated_nosql/results.json:
5 diet records
{'Diet type': 'dash', 'Protein(g)': 69.28227507163324, 'Carbs(g)': 160.5357535816619, 'Fat(g)': 101.15056160458454}
{'Diet type': 'keto', 'Protein(g)': 101.26650793650794, 'Carbs(g)': 57.970575396825396, 'Fat(g)': 153.1163558201058}
{'Diet type': 'mediterranean', 'Protein(g)': 101.11231602966345, 'Carbs(g)': 152.9055447803765, 'Fat(g)': 101.41613804905874}
{'Diet type': 'paleo', 'Protein(g)': 88.67476452119308, 'Carbs(g)': 129.55212715855572, 'Fat(g)': 135.66902668759812}
{'Diet type': 'vegan', 'Protein(g)': 56.15703022339028, 'Carbs(g)': 254.00419185282524, 'Fat(g)': 103.29967805519053}
```

Azurite Blob Storage running (healthy on 127.0.0.1:10000), `All_Diets.csv` uploaded (702,514 bytes), function processing it from Azurite and storing 5 diet documents in `simulated_nosql/results.json`. Clock: Fri Oct 2 09:59.

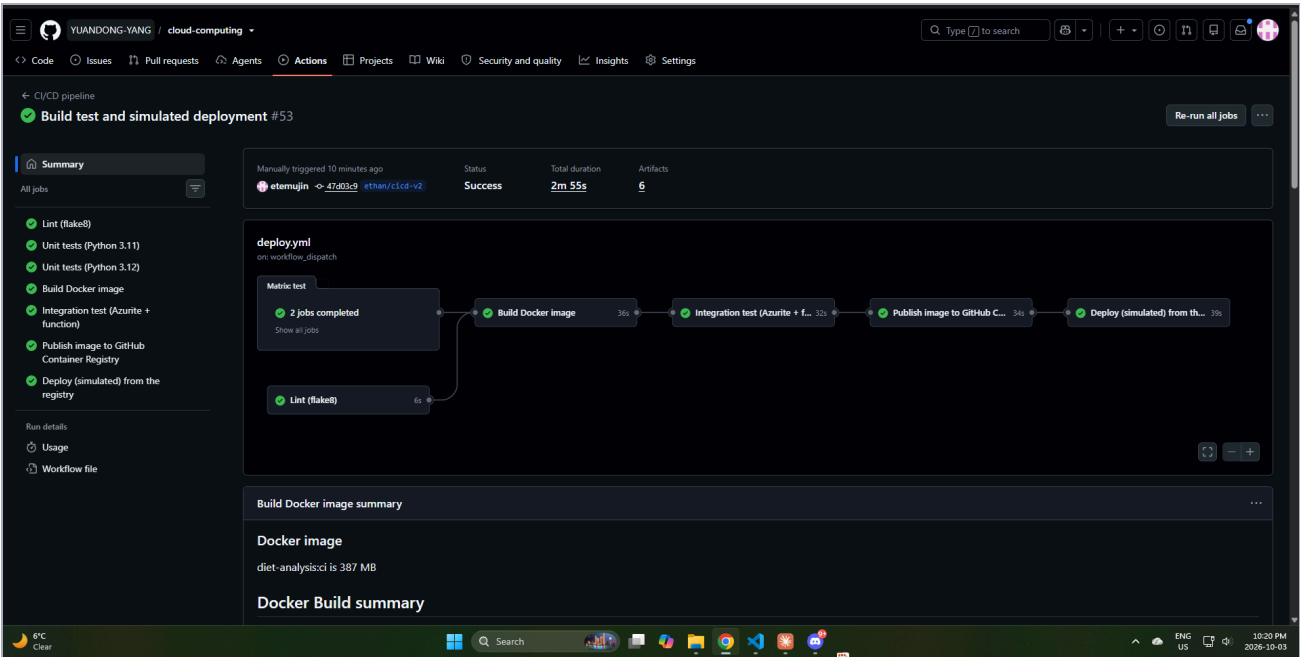
```
Activities Terminal Wed Sep 30 16:53:06
Terminal
CPSY 300 Project 1 | Task 3 | Saved upload and function logs
2026-09-30T16:53:04-06:00
Container nutritional-insights-azurite-1 Running
Container nutritional-insights-azurite-1 Waiting
Container nutritional-insights-azurite-1 Healthy
Container nutritional-insights-upload-run-cffeba3a8e5a Creating
Container nutritional-insights-upload-run-cffeba3a8e5a Created
2026-09-30T22:50:56.384987+00:00 Uploaded datasets/All Diets.csv: 702514 bytes
Container nutritional-insights-function-run-70cb92daal35 Creating
Container nutritional-insights-function-run-70cb92daal35 Created
2026-09-30T22:50:58.665879+00:00 Processed 7806 recipes from datasets/All_Diets.csv; stored 5 diet documents in simulated_nosql/results.json

NoSQL JSON documents (compact display):
{'Diet_type': 'dash', 'Protein(g)': 69.28227507163324, 'Carbs(g)': 160.5357535816619, 'Fat(g)': 101.15056160458454}
{'Diet_type': 'keto', 'Protein(g)': 101.26650793650794, 'Carbs(g)': 57.970575396825396, 'Fat(g)': 153.1163558201058}
{'Diet_type': 'mediterranean', 'Protein(g)': 101.11231602966345, 'Carbs(g)': 152.9055447803765, 'Fat(g)': 101.41613804905874}
{'Diet_type': 'paleo', 'Protein(g)': 88.67476452119308, 'Carbs(g)': 129.55212715855572, 'Fat(g)': 135.66902668759812}
{'Diet_type': 'vegan', 'Protein(g)': 56.15703022339028, 'Carbs(g)': 254.00419185282524, 'Fat(g)': 103.29967805519053}
```

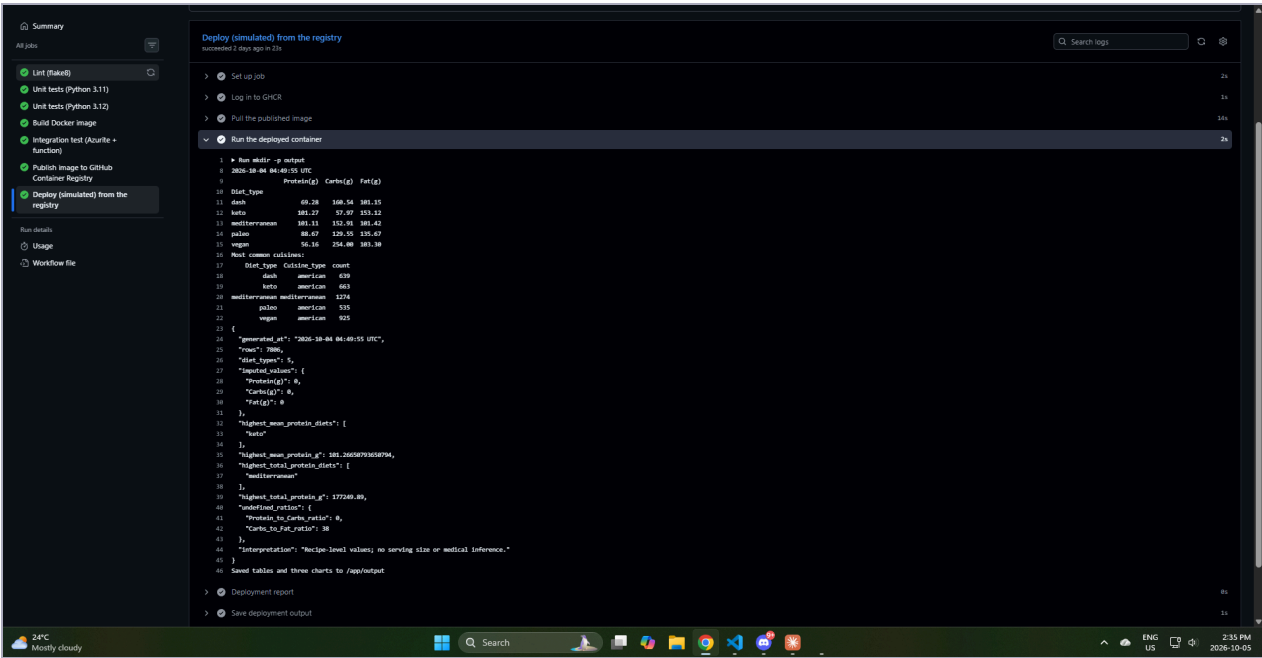
Upload and function logs with the results saved in the simulated NoSQL store (the five JSON documents shown). Clock: Wed Sep 30 16:53:06.

# Task 4: CI/CD Pipeline with GitHub Actions

Deliverable submitted in the zip: `.github/workflows/deploy.yml` · Prepared by Ethan Bayarsaikhan



Successful GitHub Actions run #53: lint and unit tests (Python 3.11 and 3.12) in parallel, then Docker build, integration test with Azurite, publish to GitHub Container Registry and simulated deploy, all green. Taskbar: 2026-10-03 10:20 PM.



Evidence of simulated deployment (run #54 on main, after the merge): the deploy job pulled the published image from ghcr.io and ran it; the running container's output shows the averages per diet, the most common cuisines and the summary (container clock 2026-10-04 04:49:55 UTC). Taskbar: 2026-10-05 2:35 PM.

Conversation 0 Commits 6 Checks 8 Files changed 1 +225 -43

eternujin commented 21 minutes ago Collaborator ...

lint + test (Python 3.11/3.12, parallel) -> build -> integration -> publish -> deploy

- build: Docker image built once with layer caching, smoke-tested, handed to later jobs as an artifact
- integration: Task 1 + Task 3 flow (Azure upload, function run) against that exact image; batch and function results verified equal
- publish: pushes the tested image to ghcr.io (sha + latest tags) using GITHUB\_TOKEN only on main or manual runs; packageswrite on this job only
- deploy: pulls the published image as a "server" would, runs it, writes a results table to the run summary; recorded as the simulated-local environment

eternujin added 6 commits 2 hours ago

- cicd\_parallel\_lint\_and\_mult\_test\_jobs
- Add build job: cached Docker build
- Add check job: azurite against build image
- Add publish job: push the tested image to GHCR with least privilege p...
- Add deploy job: give publish and deploy a working directory that exists
- Give the publish job a working directory that exists without checkout

eternujin merged commit 57aa24a into main 9 minutes ago 15 checks passed View details Revert

This branch was successfully deployed 1 active deployment Show environments

Reviews: No reviews

Assignees: No one—assign yourself

Labels: None yet

Projects: None yet

Milestone: No milestone

Development: Successfully merging this pull request may close these issues.

Notifications: Unsubscribe You're receiving notifications because you modified the open/close state.

1 participant

Lock conversation

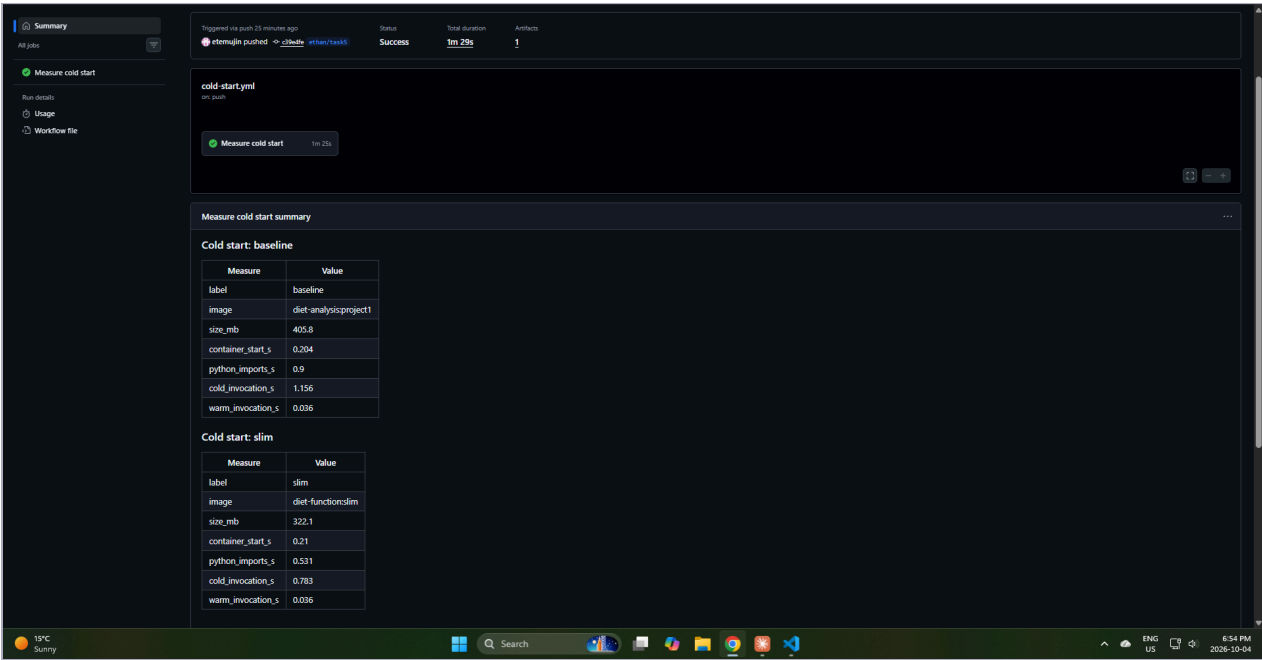
Taskbar: 2026-10-03 10:56 PM

Pipeline merged to main with 15 checks passed; GitHub records the deployment to the simulated-local environment. Taskbar: 2026-10-03 10:56 PM.

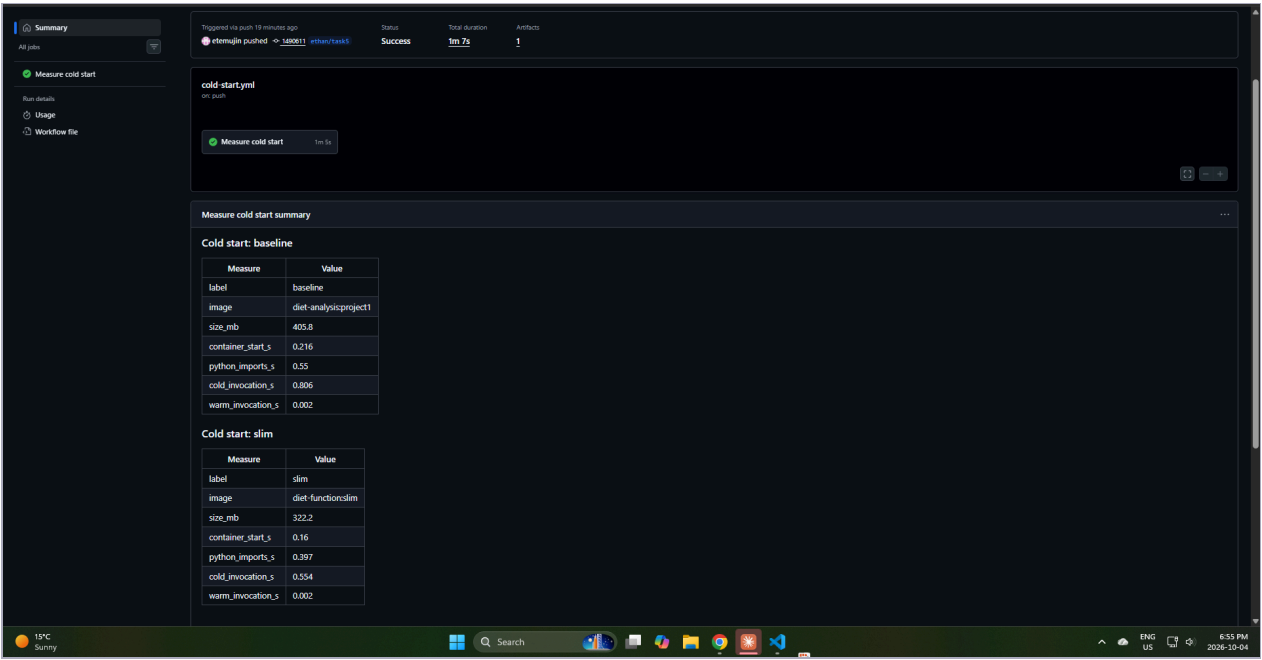
# Task 5: Enhancement Report

Deliverable: the 1-page report on the next page · Prepared by Ethan Bayarsaikhan

The screenshots below are the benchmark runs that produced the report's numbers.



Cold start benchmark run #2 (before the warm path): baseline and slim image measurements. Taskbar: 2026-10-04 6:54 PM.



Cold start benchmark run #3 (after the warm path): baseline and slim image measurements. Taskbar: 2026-10-04 6:55 PM.

## Task 5 Enhancement Report: Cold Start and Image Size

Ethan Bayarsaikhan (000961411) · CPSY 300 Project 1 · 2026-10-04 · Applied to Task 3 (serverless function) and Task 2 (container image)

### 1. Options chosen

- **Option 1, smaller and more efficient images:** a separate, function-only container image.
- **Option 2, cold start and execution speed:** reuse work across warm invocations of the function.

### 2. Research

- **Where cold-start time goes.** A cold invocation must fetch the image, start a container, start Python and import libraries, then do the work. A warm instance keeps its process alive, so module-level state survives between calls. I measured each stage before changing anything.
- **Smaller images.** Docker's build best practices advise against installing packages an image does not need. The function never draws charts, yet it shared an image with matplotlib and seaborn.
- **Reusing clients.** Microsoft's guidance on managing connections in Azure Functions recommends creating clients once and reusing them across invocations instead of per call.
- **ETags.** Azure Blob Storage gives every blob an ETag that changes whenever the blob is overwritten, so an unchanged ETag means unchanged content and the previous result is still valid.

### 3. Improvements applied

- **A: Function-only image** (`Dockerfile.function`, `requirements-function.txt`): only numpy, pandas and azure-storage-blob; only the three files the function imports; `pip --no-cache-dir`; modules precompiled with `compileall`; non-root user.
- **B: Warm path** (`lambda_function.py`): one Blob client per process, and a check of the CSV's ETag before downloading. If it matches the last processed version, the stored result is returned without downloading or recomputing. The ETag saved is the one from the download itself, so a CSV replaced mid-call is still picked up. Three new unit tests cover the cache hit, a changed CSV and a deleted results file.
- **Measurement:** `cold-start.yml` workflow with `cold_start_benchmark.py`. Each measure is the median of 5 runs after one discarded run, on a GitHub Actions runner against Azurite.

### 4. Results and impact

| Median of 5 runs                       | Run #2 (before B): baseline → slim | Run #3 (after B): baseline → slim       |
|----------------------------------------|------------------------------------|-----------------------------------------|
| Image size                             | 405.8 → 322.1 MB (−21%)            | 405.8 → 322.2 MB (−21%)                 |
| Container start                        | 0.204 → 0.210 s                    | 0.216 → 0.160 s                         |
| Python start and imports               | 0.900 → 0.531 s                    | 0.550 → 0.397 s                         |
| <b>Cold invocation</b>                 | <b>1.156 → 0.783 s (−32%)</b>      | <b>0.806 → 0.554 s (−31%)</b>           |
| <b>Warm invocation</b> (CSV unchanged) | 0.036 → 0.036 s                    | <b>0.002 → 0.002 s (−94% vs run #2)</b> |

- **A:** the image is 21% smaller, so less to download on every cold start and deployment, and cold invocations were 31–32% faster than the baseline image in both runs. Imports were also faster in the slim image; I did not isolate the cause.
- **B:** warm invocations dropped from 36 ms to about 2 ms in both images, because repeat calls make one metadata request instead of downloading and processing 7,806 rows. Cold invocations are unaffected by design: a new process starts with an empty cache.
- **Limits:** the same baseline image measured 1.156 s and 0.806 s in two runs, so shared CI runners are noisy; I compare images only within the same run. These are local simulations, not real Azure Functions, where image download time would make A matter more. Related: Docker layer caching in the CI pipeline cut the image build from 61 s to 9 s (runs #47 and #48).

Sources: Docker, "Building best practices" ([docs.docker.com/build/building/best-practices](https://docs.docker.com/build/building/best-practices)) · Microsoft Learn, "Manage connections in Azure Functions" ([learn.microsoft.com/azure/azure-functions/manage-connections](https://learn.microsoft.com/azure/azure-functions/manage-connections)) · Microsoft Learn, "Managing concurrency in Blob Storage" ([learn.microsoft.com/azure/storage/blobs/concurrency-manage](https://learn.microsoft.com/azure/storage/blobs/concurrency-manage)) · Python docs, "compileall" ([docs.python.org/3/library/compileall.html](https://docs.python.org/3/library/compileall.html)) · Evidence: Cold start benchmark runs #2 and #3, [docs/evidence/task5-benchmark-before.png](#) and [-after.png](#).